

Travaux Pratiques

Programmation Système en C

Analyse Comparée : Exécution Séquentielle vs. Multitâche

Mots-clés : appels système bloquants, `sleep()`, compilation séparée, threads POSIX (`pthread`), parallélisme, mesure de performance (`time`)

École Normale Supérieure d'Enseignement Technique (ENSET) de Douala

Département de Génie Informatique (GINFO) – Niveau II4

Module : Systèmes à Microcontrôleurs

Objectifs du TP

Objectifs pédagogiques

À l'issue de ce TP, l'étudiant doit être capable de :

- Comprendre l'impact des appels système bloquants (`sleep()`) sur le temps d'exécution d'un processus ;
- Maîtriser la compilation séparée sous Linux avec `gcc` (édition de liens multi-fichiers) ;
- Mettre en œuvre le parallélisme de threads via la bibliothèque POSIX `pthread` ;
- Analyser et interpréter les performances temporelles à l'aide de la commande `time` ;
- Comparer rigoureusement une exécution séquentielle et une exécution concurrente sur un même problème.

Démarche méthodologique

La démarche de ce TP suit trois étapes :

1. **Observer** une exécution séquentielle (monothread) et mesurer son coût temporel ;
2. **Transformer** le programme en version multithread (`pthread`) ;
3. **Comparer** les deux mesures de temps et expliquer le gain de performance obtenu.

Pour chaque exercice, vous répondrez aux questions en argumentant systématiquement vos réponses à l'aide du code fourni.

1 Exercice 1 : Analyse d'une Exécution Séquentielle (Monothread)

Rappel de cours

Un appel à `sleep(n)` suspend l'exécution du processus appelant pendant `n` secondes. Pendant cette suspension, le processus est placé par l'ordonnanceur (*scheduler*) du noyau dans l'état **bloqué** : il ne consomme aucun cycle CPU, le processeur étant alors disponible pour d'autres tâches. Dans un programme **séquentiel**, les instructions s'exécutent les unes après les autres, dans l'ordre où elles sont écrites : le processeur ne passe à l'instruction suivante que lorsque la précédente est totalement terminée. Par conséquent, tout appel bloquant retarde mécaniquement l'exécution de **toutes** les instructions qui le suivent, qu'elles dépendent de lui ou non.

Cette caractéristique est essentielle à comprendre : dans l'Exercice 1, les fonctions `somme` et `produit` n'ont **aucun lien logique** entre elles (le résultat de l'une ne dépend pas de l'autre), mais elles s'exécutent malgré tout l'une après l'autre, simplement parce que le programme est écrit de façon séquentielle. C'est précisément cette contrainte que l'Exercice 2 lèvera grâce aux threads.

Glossaire – Mots-clés à connaître

Processus	Instance d'un programme en cours d'exécution, disposant de son propre espace mémoire et de ses propres ressources (fichiers ouverts, registres, etc.), gérée par le système d'exploitation.
Appel système	Mécanisme par lequel un programme demande un service au noyau du système d'exploitation (par exemple : lire un fichier, attendre, écrire à l'écran). L'exécution bascule alors temporairement du mode utilisateur vers le mode noyau.
Appel bloquant	Appel système qui suspend l'exécution du processus appelant jusqu'à ce qu'une condition soit remplie (ici, l'écoulement d'un délai). Le processus ne progresse plus tant que l'appel n'est pas terminé.
<code>sleep(n)</code>	Fonction de la bibliothèque standard C qui met en pause le processus appelant pendant <code>n</code> secondes, en le plaçant dans l'état bloqué.
Ordonnanceur (<i>scheduler</i>)	Composant du noyau qui décide, à chaque instant, quel processus ou thread doit utiliser le processeur. Un processus bloqué est retiré de la file des processus exécutables.
Exécution séquentielle	Mode d'exécution dans lequel les instructions d'un programme sont traitées strictement les unes après les autres, sans chevauchement temporel.
Compilation séparée	Technique consistant à compiler indépendamment plusieurs fichiers source (.c) en fichiers objets, puis à les assembler (édition de liens) en un seul exécutable. Elle permet de structurer un projet en modules réutilisables et de ne recompiler que les fichiers modifiés.
Garde d'en-tête (<i>include guard</i>)	Mécanisme de préprocesseur (<code>#ifndef/#define/#endif</code>) empêchant

l'inclusion multiple d'un même fichier d'en-tête lors de la compilation.

Édition de liens (*linking*)

Étape finale de la compilation qui assemble les différents fichiers objets (.o) et les bibliothèques nécessaires pour produire un exécutable unique.

Temps réel (real)

Durée totale écoulée entre le début et la fin de l'exécution d'un programme, mesurée par une horloge externe (*wall-clock time*).

Temps utilisateur (user)

Temps CPU passé à exécuter les instructions du programme lui-même, en mode utilisateur.

Temps système (sys)

Temps CPU passé par le noyau à traiter les appels système effectués pour le compte du processus.

On considère une application modulaire composée de deux fonctions principales : **somme** et **produit**. Chacune de ces fonctions simule un traitement long en alternant calculs et pauses via la fonction `sleep()`.

Organisation des fichiers

Listing 1: Arborescence du projet

```
projet/  
|-- include/  
|   '-- fonctions.h      // Declarations des prototypes  
|-- src/  
|   |-- mono.c           // Programme principal (sequentiel)  
|   |-- somme.c           // Definition de la fonction somme()  
|   '-- produit.c        // Definition de la fonction produit()  
 '-- bin/                // Executables generes
```

Extrait du fichier d'en-tête fonctions.h

```
1 #ifndef FONCTIONS_H  
2 #define FONCTIONS_H  
3  
4 void somme(int a, int b);  
5 void produit(int a, int b);  
6  
7 #endif
```

Listing 2: fonctions.h

Extrait du fichier principal mono.c

```
1 #include <stdio.h>  
2 #include "fonctions.h"  
3  
4 int main(void) {  
5     somme(3, 4);  
}
```

```
6   produit(3, 4);  
7   return 0;  
8 }
```

Listing 3: mono.c – exécution strictement séquentielle

Comportement attendu de somme et produit

- La fonction `somme` effectue un `sleep(10)` avant le calcul, puis un `sleep(5)` après l’affichage du résultat.
- La fonction `produit` effectue un `sleep(5)` avant le calcul, puis un `sleep(10)` après l’affichage du résultat.

Questions

Question 1 – Structure et Prototypes.

Expliquez le rôle des directives de préprocesseur `#ifndef FONCTIONS_H` et `#define FONCTIONS_H` dans un fichier d’en-tête.

Question 2 – Analyse du Code Séquentiel.

Le programme principal (`mono.c`) appelle successivement `somme(3, 4)` puis `produit(3, 4)`, selon le comportement décrit ci-dessus. Calculez le temps théorique total d’attente passive de ce programme. Détaillez votre calcul.

Question 3 – Compilation Séparée.

Donnez la commande `gcc` permettant de compiler ensemble les fichiers sources `src/mono.c`, `src/somme.c` et `src/produit.c` afin d’obtenir un exécutable nommé `bin/mono`. Expliquez brièvement le rôle de l’option `-o`.

Question 4 – Mesure de Temps.

Lors de l’exécution de la commande `time ./bin/mono`, le terminal renvoie les informations suivantes :

```
real    0m30,007s  
user    0m0,000s  
sys     0m0,002s
```

Définissez les trois métriques (`real`, `user`, `sys`) et interprétez la valeur quasi-nulle des temps CPU (`user` et `sys`).

2 Exercice 2 : Optimisation par le Multithreading (pthread)

Rappel de cours

Un **thread** (ou fil d'exécution) est une unité d'exécution légère au sein d'un même processus, partageant le même espace mémoire (variables globales, tas, fichiers ouverts) avec les autres threads du même processus. Contrairement à un programme purement séquentiel, plusieurs threads peuvent être ordonnancés **de façon concurrente** : lorsqu'un thread est bloqué (par exemple en attente d'un `sleep()`), le processeur peut être attribué à un autre thread prêt à s'exécuter, plutôt que de rester inactif.

Il est important de distinguer deux notions souvent confondues :

- la **concurrency** : plusieurs tâches progressent sur une période de temps commune, en s'entrelaçant (c'est le cas ici, même sur un processeur à un seul cœur) ;
- le **parallélisme** : plusieurs tâches s'exécutent réellement au même instant, sur des cœurs CPU distincts.

Dans cet exercice, le gain de performance provient essentiellement de la concurrence : pendant qu'un thread est endormi (donc inactif), l'autre thread peut progresser, même sur un seul cœur. La bibliothèque POSIX **pthread** (*POSIX threads*) fournit l'interface standard de création et de gestion des threads sous Linux.

Glossaire – Mots-clés à connaître

Thread (fil d'exécution)

Unité d'exécution légère au sein d'un processus. Plusieurs threads d'un même processus partagent le même espace mémoire mais possèdent chacun leur propre pile d'exécution et leur propre compteur d'instructions.

Bibliothèque POSIX pthread

Ensemble standardisé de fonctions C (`pthread_create`, `pthread_join`, etc.) permettant de créer et synchroniser des threads sur les systèmes compatibles POSIX (Linux, macOS, etc.).

Concurrence Capacité de plusieurs tâches à progresser sur une même période de temps, en s'entrelaçant, sans nécessairement s'exécuter au même instant exact.

Parallélisme Exécution réellement simultanée de plusieurs tâches, sur des unités de calcul (cœurs) distinctes.

pthread_create Fonction qui crée un nouveau thread et démarre, de façon asynchrone, l'exécution d'une fonction donnée dans ce nouveau thread.

pthread_join Fonction qui bloque le thread appelant jusqu'à la terminaison complète du thread désigné en argument. Permet de attendre la fin d'un thread.

Pointeur générique void*

Pointeur ne précisant aucun type de donnée particulier ; il peut référencer n'importe quelle structure de données, à condition d'être transtypé (*cast*) vers le type réel avant utilisation.

Transtypage (*cast*)

Conversion explicite d'une valeur d'un type vers un autre type,

	par exemple <code>(int*) args</code> , pour indiquer au compilateur comment interpréter une zone mémoire.
Thread-safe	Qualifie une fonction ou une portion de code pouvant être exécutée simultanément par plusieurs threads sans provoquer d'erreur ou de corruption de données.
Tampon (<i>buffer</i>) de sortie	Zone mémoire intermédiaire dans laquelle les caractères à afficher sont temporairement stockés avant d'être réellement envoyés au terminal.
<code>fflush(stdout)</code>	Fonction qui force le vidage immédiat du tampon de sortie standard, garantissant un affichage immédiat plutôt que différé.

Afin de réduire la durée totale de l'application, on souhaite exécuter les fonctions `somme` et `produit` de manière concurrente en utilisant des threads POSIX.

Extrait du fichier `multi.c`

```

1 #include <stdio.h>
2 #include <pthread.h>
3
4 void* somme(void* args) {
5     int* ab = (int*) args;
6     sleep(10);
7     int s = ab[0] + ab[1];
8     printf("Somme = %d\n", s);
9     fflush(stdout);
10    sleep(5);
11    return NULL;
12 }
13
14 void* produit(void* args) {
15     int* ab = (int*) args;
16     sleep(5);
17     int p = ab[0] * ab[1];
18     printf("Produit = %d\n", p);
19     fflush(stdout);
20     sleep(10);
21     return NULL;
22 }
23
24 int main(void) {
25     pthread_t t1, t2;
26     int ab[2] = {3, 4};
27
28     pthread_create(&t1, NULL, somme, ab);
29     pthread_create(&t2, NULL, produit, ab);
30
31     pthread_join(t1, NULL);
32     pthread_join(t2, NULL);
33
34     return 0;
35 }

```

Listing 4: `multi.c` – exécution concurrente avec pthread

Questions

Question 1 – Adaptation des Prototypes.

Pourquoi les fonctions destinées à être lancées par `pthread_create` doivent-elles obligatoirement posséder la signature `void* nom_fonction(void* args)` ? Comment récupère-t-on, dans le corps des fonctions, les paramètres entiers passés en argument ?

Question 2 – Cycle de Vie des Threads.

Dans le fichier `multi.c`, identifiez et expliquez le rôle des deux fonctions fondamentales de la bibliothèque `pthread` utilisées pour :

- initialiser et démarrer l'exécution des tâches en parallèle ;
- bloquer le thread principal jusqu'à la fin de l'exécution des tâches de calcul.

Question 3 – Compilation Multithread.

Écrivez la ligne de commande `gcc` nécessaire pour compiler le fichier `src/multi.c` en un exécutable `bin/multi`. Quelle option spécifique est indispensable ici, et quels sont ses rôles ?

Question 4 – Analyse des Performances.

Après exécution concurrentielle, la commande `time ./bin/multi` affiche un temps `real` de `0m15,026s`.

- Justifiez rigoureusement ce passage de 30 secondes à 15 secondes en détaillant l'enchaînement chronologique des événements (vous pouvez vous appuyer sur une frise temporelle).
- Pourquoi l'ajout de l'instruction `fflush(stdout)` après chaque `printf` est-il une bonne pratique lors de l'affichage textuel au sein de threads concurrents ?

Démarche méthodologique

Pour bien répondre à la Question 4 : construisez une frise temporelle (axe horizontal = temps écoulé en secondes) sur laquelle vous positionnez, pour chaque thread, les phases de `sleep`, de calcul et d'affichage. Identifiez ensuite l'instant où *chaque* thread se termine, puis l'instant où *les deux* threads sont terminés (le `main` ne reprend la main qu'après les deux `pthread_join`).